



Department of Computer Science
DATABASE SYSTEMS — COMP333
Phase 4

Birzeit Library Database System



Prepared by:

Student1: Zainab Murad

ID: 1211020

Student2: Asmaa Jaraba

ID: 1231081

Course instructor: Ihab Alshaer

Abstract

This project implements a database-driven sales system for BZU Library Store, enabling students to browse products (books, stationery, services, and e-books), add items to a cart, place orders, and track order status. The system also supports administrative functions for managing products and categories and monitoring orders. The database was designed using an entity–relationship (ER) model, mapped to a relational schema, and implemented in MySQL with integrity constraints (primary keys, foreign keys, and uniqueness constraints). The report documents the project requirements and representative SQL queries, presents the conceptual ER design and its conversion to relations, and provides normalization analysis demonstrating that the final relations satisfy Third Normal Form (3NF) with justification. The goal is to ensure correctness, reduce redundancy, and support reliable transactional operations and reporting queries.

Table of Contents

Abstract	(I)
1. Introduction	(1)
2. Project Overview and Main Functionality	(1)
2.1 System Scope	(1)
2.2 User Roles and Permissions	(1)
3. Conceptual Design (ER Model)	(2)
3.1 Entity Sets (with Key Attributes)	(2)
3.2 Relationship Sets and Cardinality	(3)
3.3 ER Diagrams (Before and After Resolving M:N)	(4)
4. Queries (Sentence Form)	(6)
4.1 branches	(6)
4.2 product and catalog	(6)
4.3 Cart as “Pending Order”	(6)
4.4 orders	(7)
4.5 order items	(7)
5. Normalization	(8)
5.1 Third Normal Form (3NF) Verification Summary	(8)
5.2 First Normal Form (1NF) Verification	(10)
5.3 Second Normal Form (2NF) Verification	(11)
6. Conclusion	(12)
7. References	(13)

Table of Figures

Figure 1: ER diagram before resolving the Orders–Products M:N relationship(4)

Figure 2: Final ER diagram after resolving the Orders–Products M:N relationship ...(5)

1. Introduction

The BZU Library Store project provides a structured database solution for managing university-scale retail transactions. By supporting product listings and order management through relational constraints, the system ensures reliable data handling for campus services.

Following standard technical reporting practices, this document outlines the project scope, the conceptual ER model, and the MySQL-based relational schema. It further demonstrates system utility through targeted SQL queries and justifies the database architecture through a formal normalization analysis to 3NF.

2. Project Overview and Main Functionality

2.1 System Scope

The BZU Library Store is a specialized relational database application designed to streamline retail operations within a university environment. The system serves as a centralized management hub, maintaining comprehensive records for user accounts, a categorized product catalog, and a robust transaction history. By utilizing relational constraints, the system ensures that every order is accurately linked to a specific user and contains detailed line-items that capture the quantity and unit price of products at the time of purchase. This architecture provides the foundation for a reliable point-of-sale environment that can handle frequent, small-scale transactions while maintaining high data integrity.

2.2 User Roles and Permissions

The application operates under a role-based access model, distinguishing between students and administrative staff to ensure secure and efficient workflows. For the student

user, the system prioritizes a seamless purchasing journey. Students are empowered to manage their own profiles through registration and login, browse the inventory via category filters, and manage a dynamic shopping cart. The transition from browsing to purchasing is handled through a structured checkout process, after which students can access a permanent historical record of their past transactions and itemized order details.

Administrative users are provided with elevated permissions to oversee the store's backend operations and maintain the integrity of the catalog. Their primary responsibilities include managing product availability and categorizing inventory to ensure the store remains up to date. Beyond inventory management, administrators possess the authority to orchestrate the order lifecycle, transitioning orders through various fulfillment stages such as processing and completion. To support data-driven decision-making, the system allows administrators to generate operational summaries and reports, providing critical insights into sales totals and inventory counts through aggregate data analysis.

3. Conceptual Design

3.1 Entity sets (with key attributes)

- 1. USERS:** id (PK), full_name, email (unique), password, role, status, created_at
- 2. CATEGORIES:** id (PK), name (unique)
- 3. PRODUCTS:** id (PK), name, description, price, product_type, image_filename, is_available, created_at
- 4. ORDERS:** id (PK), order_date, status, total_price, notes
- 5. ORDER_ITEMS** (associative entity for the M:N relationship), order_id (part of PK, FK), product_id (part of PK, FK), quantity, unit_price

6. BRANCHES: id (PK), name, address, phone, manager_name, open_time, close_time, is_active, monthly_sales, staff_count, created_at

3.2 Relationship Sets and Cardinality

The conceptual integrity of the BZU Library Store is defined by the cardinality and structural constraints of its relationship sets. The connection between **USERS** and **ORDERS** is defined by a one-to-many (1:N) relationship, indicating that while each order is uniquely attributed to a single user for tracking and billing purposes, an individual user may place multiple orders over time.

In addition, the system supports multiple service points through the **BRANCHES** entity. The relationship between **BRANCHES** and **ORDERS** is also one-to-many (1:N), meaning that each order is associated with exactly one branch (for processing and management purposes), while each branch can handle many orders over time.

Similarly, a one-to-many (1:N) relationship exists between **CATEGORIES** and **PRODUCTS**; this hierarchical structure ensures that every product is organized under exactly one category, such as “Stationery” or “Textbooks,” while a single category can include many different products.

The relationship **contains** between **ORDERS** and **PRODUCTS** is many-to-many (M:N): one order can include many products, and one product can appear in many orders. Additionally, this relationship has its own descriptive attributes (**quantity**, **unit_price**) that depend on the specific order–product combination. Therefore, it is modeled using a separate associative entity, **ORDER_ITEMS**, to represent relationship instances clearly, maintain data integrity, and avoid redundancy. Consequently, the final ER diagram after resolving the Orders–Products many-to-many relationship using **ORDER_ITEMS** is presented in the figure below.

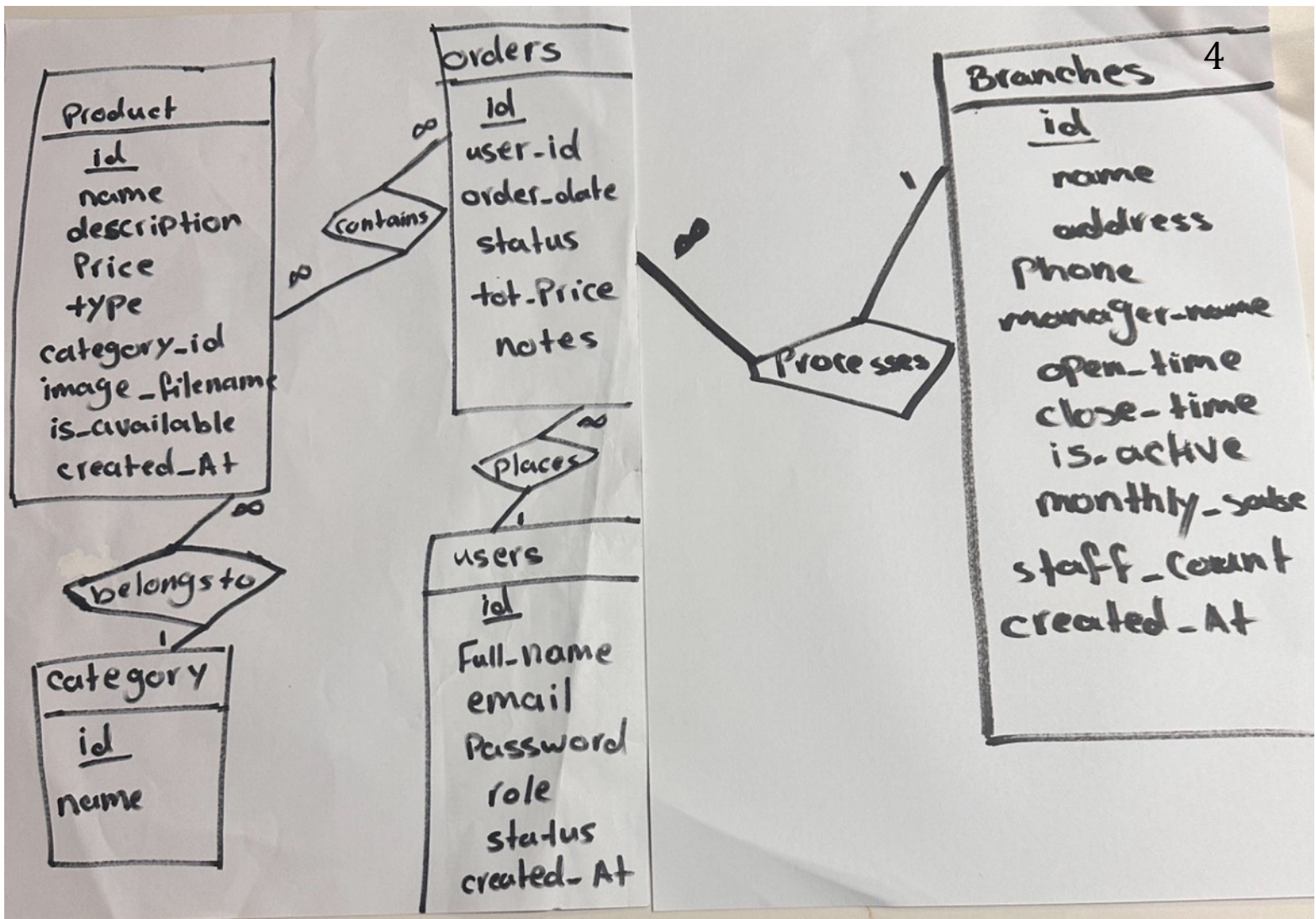


Figure 1: ER diagram before resolving the Orders–Products M:N relationship

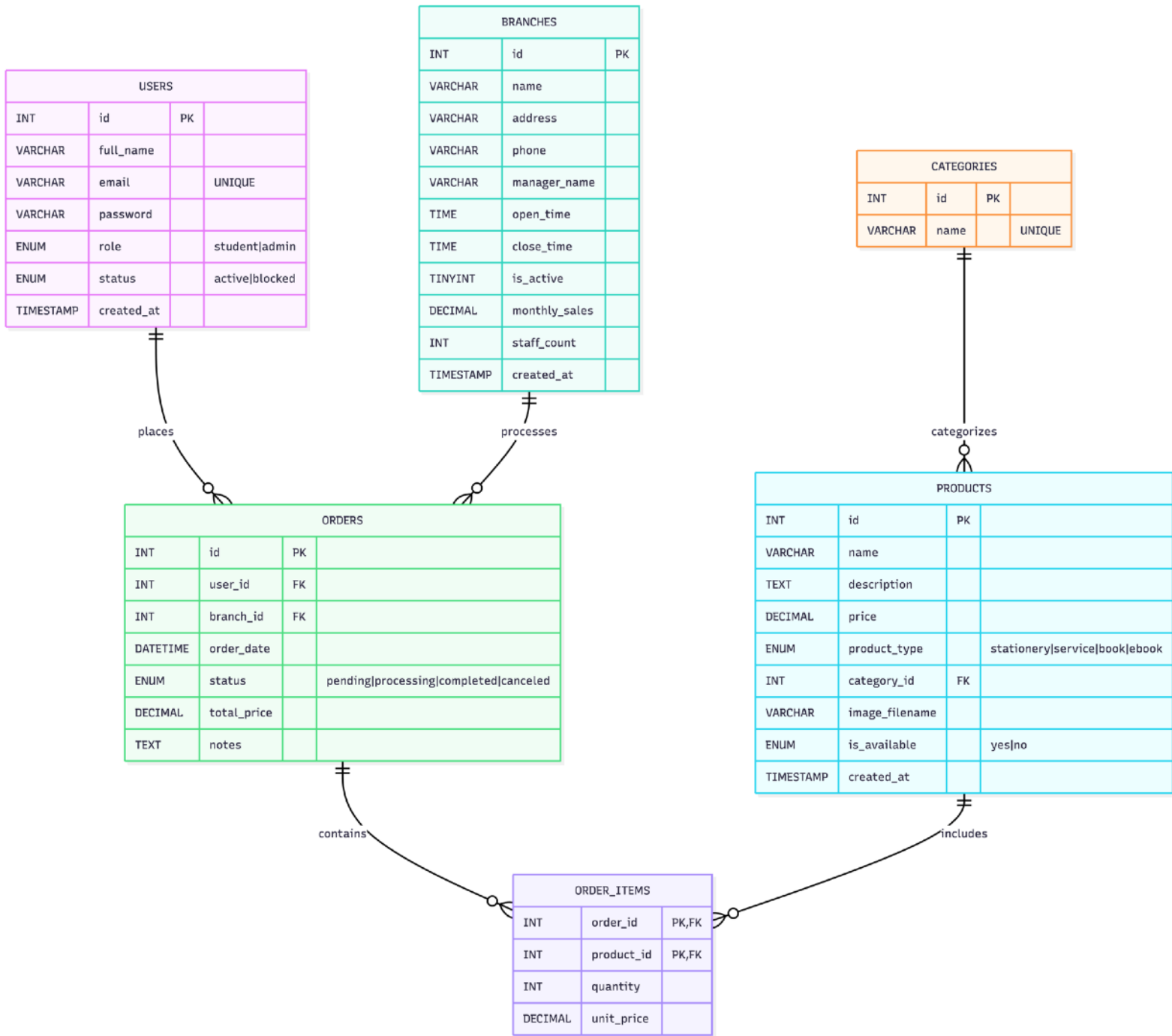


Figure 2: Final ER diagram after resolving the Orders–Products M:N relationship (ORDER_ITEMS)

4. Queries

4.1 Branches

1. Retrieve branch names and their total monthly sales for branches whose total monthly sales exceed 3000, sorted by total sales (highest first).

4.2 Products & Catalog

1. Retrieve the ID, name, and price of all products that are of type book.
2. Retrieve product ID, name, price, product type, and category name for all products, sorted by newest product first.
3. Retrieve product ID, name, and price for all products, sorted by price ascending.
4. Retrieve product ID, name, and price for all products, sorted by price descending.
5. Retrieve the minimum price among all products.
6. Retrieve the maximum price among all products.

4.3 Cart (Pending Order)

1. Retrieve the total cart value (sum of quantity $\times$ unit price) for the most recent pending order of a specific user (e.g., user ID = 2).
2. Retrieve the pending order (cart) details for a specific user (e.g., user ID = 2), including: order ID, status, order date, each product name, quantity, unit price, and the line total (quantity $\times$ unit price).

4.4 Orders

10. 1. Retrieve the order ID, status, and order date for all orders of a specific user (e.g., user ID = 2), sorted by newest order first.
11. 2. Retrieve all orders for a specific user (e.g., user ID = 1) including status, total price, order date, and notes, sorted by newest first.
12. 3. Retrieve each order ID, customer full name, order status, order date, and total price for all orders, sorted by newest orders first.
13. 4. Retrieve each order ID and its computed total (sum of quantity $\times$ unit price) for orders whose computed total is greater than 100, sorted by total descending.

4.5 Order Items

14. 1. Retrieve the product name, quantity, and unit price for all items in a specific order (e.g., order ID = 18).

5. Normalization

5.1 3NF Verification Summary Table

Relation	Candidate Key(s)	Main Functional Dependencies (FDs)	3NF?	Justification
BRANCHES(id, name, address, phone, manager_name, open_time, close_time, is_active, monthly_sales, staff_count, created_at)	{id}	id $\rightarrow$ name, address, phone, manager_name, open_time, close_time, is_active, monthly_sales, staff_count, created_at	Yes	All non-key attributes depend on branch id. No transitive dependency among non-key attributes is assumed in the stored design.
USERS(id, full_name, email, password, role, status, created_at)	{id}, {email}	id $\rightarrow$ full_name, email, password, role, status, created_at email $\rightarrow$ id, full_name, password, role, status, created_at	Yes	Every non-trivial FD has a determinant that is a candidate key (superkey), so no transitive dependency exists.
CATEGORIES(id, name)	{id}, {name}	id $\rightarrow$ name name $\rightarrow$ id	Yes	Determinants are candidate keys; therefore the relation satisfies 3NF.
PRODUCTS(id, name, description, price, product_type, category_id, image_filename, is_available, created_at)	{id}	id $\rightarrow$ name, description, price, product_type, category_id, image_filename, is_available, created_at	Yes	All non-key attributes depend on product id. Category descriptive data is stored in CATEGORIES, preventing transitive dependencies.

ORDERS(id, user_id, branch_id, order_date, status, total_price, notes)	{id}	id $\rightarrow$ user_id, branch_id, order_date, status, total_price, notes	Yes	All non-key attributes depend on order id. User and branch descriptive data are stored in USERS and BRANCHES, avoiding transitive dependencies via user_id or branch_id.
--	------	---	-----	--

Conclusion

All relations satisfy Third Normal Form (3NF) because, for each relation, every non-trivial functional dependency either has a superkey determinant or does not introduce any transitive dependency among non-key attributes.

5.2 First Normal Form (1NF) Verification

Verification item	What to check	Evidence in our schema	Conclusion
Atomic values (no multi-valued attributes)	Each column stores a single value per row (no lists, sets, arrays, or comma-separated values).	All attributes use single-valued types (INT, VARCHAR, DECIMAL, DATETIME/TIMESTAMP). No attribute stores multiple values (e.g., no 'products_list' in ORDERS).	1NF satisfied.
No repeating groups / repeating columns	Avoid patterns like product1, product2, product3 or repeating sub-structures inside one tuple.	Repeating order products are modeled using ORDER_ITEMS, not repeated columns in ORDERS.	1NF satisfied.
One fact per row for repeating data	If a parent entity can have many child items, represent child items in a separate relation.	Each ORDER_ITEMS tuple represents exactly one product within one order (order_id, product_id).	1NF satisfied.

5.3 Second Normal Form (2NF) Verification

Verification item	What to check	Evidence in our schema	Conclusion
Single-key relations automatically meet 2NF	If the primary key is a single attribute, partial dependency cannot occur.	USERS, CATEGORIES, PRODUCTS, BRANCHES, and ORDERS each have a single-attribute primary key (id).	2NF satisfied for these relations.

Composite-key relation check (ORDER_ITEMS)	For composite PK (order_id, product_id), ensure non-key attributes depend on the whole key.	ORDER_ITEMS has composite PK (order_id, product_id) and non-key attributes (quantity, unit_price). Both describe a specific line item in a specific order, so (order_id, product_id) → (quantity, unit_price).	No partial dependency.
No dependency on a subset of composite key	Verify there is no order_id → quantity and no product_id → quantity/unit_price within ORDER_ITEMS.	quantity depends on the order-product pair. unit_price is stored as price-at-order-time, so it can vary per order and is not functionally determined by product_id alone.	2NF satisfied for ORDER_ITEMS; therefore the full schema is in 2NF.

6. Conclusion

The BZU Library Store project successfully delivered a comprehensive relational database solution tailored to the specific operational needs of a university retail environment. By integrating student and administrative workflows, the system facilitates essential functions such as secure account management, intuitive product discovery through categorization, and a robust transaction pipeline. The implementation of a "pending-order" status effectively manages the transition from a digital shopping cart to a finalized sale, ensuring that the lifecycle of every order is tracked with precision through the structured interaction of order and itemized records.

The technical foundation of the system is rooted in a meticulously developed Entity-Relationship (ER) model. A key highlight of this design is the effective resolution of the many-to-many relationship between orders and products via the `ORDER_ITEMS` associative entity. This approach allows the system to capture transaction-specific data, such as historical unit prices, while maintaining a clean separation of concerns. This conceptual model was seamlessly mapped into a MySQL relational schema, where the enforcement of primary keys, foreign keys, and uniqueness constraints guarantees data integrity and prevents the entry of orphaned or inconsistent records.

To ensure long-term scalability and performance, the database architecture was rigorously analyzed and verified to satisfy Third Normal Form (3NF). This level of normalization minimizes data redundancy and eliminates the risk of insertion, update, and deletion anomalies, thereby simplifying future maintenance. Ultimately, the BZU Library Store system stands as a reliable, high-integrity platform that demonstrates professional database modeling practices and provides a scalable framework capable of meeting the dynamic requirements of a campus-based storefront.

7. References

- (1) COMP333 – Database Systems Project Specification, Birzeit University, Course Handout, 2025/2026. Accessed: 17 January 2026.
- (2) COMP333 – Database Systems, Chapter 2: The Relational Model (Relational Mapping Rules), Birzeit University, Course Handout (PDF provided by instructor). Accessed: 17 January 2026.
- (3) COMP333 – Database Systems, Chapter 7: Normalization, Birzeit University, Course Handout (PDF provided by instructor). Accessed: 17 January 2026.
- (4) Mermaid Chart, “Entity Relationship Diagrams (erDiagram) Syntax,” Mermaid Documentation. Available at: **docs.mermaidchart.com** (Mermaid OSS → Syntax → Entity Relationship Diagrams). Accessed: 17 January 2026. (docs.mermaidchart.com)